

SIMD Autovectorization in Numba

SIMD

- You should have some idea of "Single Instruction, Multiple Data" (SIMD) operations
- Used numpy matrix-vector and matrix-matrix
- LLVM backend used by Numba can generate them in some cases to execute loops more quickly.
 - *low level virtual machine*
 - This process is called "autovectorization."

SIMD instructions sets

Intel processors have support for SIMD instruction sets like:

- SSE (128-bit inputs)
- AVX (256-bit inputs)
- AVX-512 (512-bit inputs, Skylake-X and later or Xeon Phi)

```
In [1]: import numpy as np
        from numba import jit
```

Autovectorisation

- Tricky to see if LLVM actually autovectorized loop
- Diagonistics from Numba evolving.
- We will make some crude code to determine it

```
In [2]: def find_instr(func, keyword, sig=0, limit=5):
        count = 0
        for l in func.inspect_asm(func.signatures[sig]).split('\n'):
            if keyword in l:
                count += 1
                print(l)
                if count >= limit:
                    break
        if count == 0:
            print('No instructions found')
```

Vector registers

On x86_64, the name of the registers used indicates which level of SIMD is in use:

- SSE: `xmmX`
- AVX/AVX2: `ymmX`
- AVX-512: `zmmX`

where X is an integer.

Note: The method we use below to find SIMD instructions will only work on Intel/AMD CPUs. Other platforms have entirely different assembly language syntax for SIMD instructions.

Basic SIMD

Let's start with a simple function that returns the square difference between two arrays, as you might write for a least-squares optimization:

```
In [3]: @jit(nopython=True)
def sqdiff(x, y):
    out = np.empty_like(x)
    for i in range(x.shape[0]):
        out[i] = (x[i] - y[i])**2
    return out
```

We shall now try it with float (`float32`) values

```
In [4]: x32 = np.linspace(1, 2, 10000, dtype=np.float32)
y32 = np.linspace(2, 3, 10000, dtype=np.float32)
sqdiff(x32, y32)
```

```
Out[4]: array([1.          , 0.99999976, 1.          , ..., 1.          , 1.00000002 ,
              1.          ], dtype=float32)
```

And double (`float64`) values

```
In [5]: x64 = x32.astype(np.float64)
y64 = y32.astype(np.float64)
sqdiff(x64, y64)
```

```
Out[5]: array([1.          , 0.99999976, 1.          , ..., 1.          , 1.000000024,
              1.          ])
```

Numba has created two different implementations of the function, one for `float32` 1-D arrays, and one for `float64` 1-D arrays:

```
In [6]: sqdiff.signatures
```

```
Out[6]: [(Array(float32, 1, 'C', False, aligned=True),
          Array(float32, 1, 'C', False, aligned=True)),
          (Array(float64, 1, 'C', False, aligned=True),
          Array(float64, 1, 'C', False, aligned=True))]
```

- This allows Numba (and LLVM) to specialize the use of the SIMD instructions for each situation.
- Lower precision floating point allows twice as many values to fit into a SIMD register.
- We will see that for the same number of elements, the `float32` calculation goes twice as fast:

```
In [7]: %timeit sqdiff(x32, y32)
%timeit sqdiff(x64, y64)
```

5.61 μ s $\pm$ 610 ns per loop (mean $\pm$ std. dev. of 7 runs, 100,000 loops each)
 8.5 μ s $\pm$ 570 ns per loop (mean $\pm$ std. dev. of 7 runs, 100,000 loops each)

We can check for SIMD instructions in both cases. (Due to the order of compilation above, signature 0 is the `float32` implementation and signature 1 is the `float64` implementation.)

```
In [8]: print('float32:')
find_instr(sqdiff, keyword='subp', sig=0)
print('----\nfloat64:')
find_instr(sqdiff, keyword='subp', sig=1)
```

```
float32:
    vsubps    (%rax,%rdi,4), %ymm0, %ymm0
    vsubps    32(%rax,%rdi,4), %ymm1, %ymm1
    vsubps    64(%rax,%rdi,4), %ymm2, %ymm2
    vsubps    96(%rax,%rdi,4), %ymm3, %ymm3
    vsubps    128(%rax,%rdi,4), %ymm0, %ymm0
----
float64:
    vsubpd    (%rax,%rdi,8), %ymm0, %ymm0
    vsubpd    32(%rax,%rdi,8), %ymm1, %ymm1
    vsubpd    64(%rax,%rdi,8), %ymm2, %ymm2
    vsubpd    96(%rax,%rdi,8), %ymm3, %ymm3
    vsubpd    128(%rax,%rdi,8), %ymm0, %ymm0
```

- In x86_64 assembly, SSE uses `subps` for "subtraction packed single precision" (AVX uses `vsubps`), representing vector float32 operations.
- The `subpd` instruction (AVX = `vsubpd`) stands for "subtraction packed double precision", representing float64 operations.

SIMD and branching

- In general, the autovectorizer cannot deal with branches inside loops
- Branches introduce unpredictability

```
In [9]: @jit(nopython=True)
def branching_code(x, y):
```

```

out = np.empty_like(x)
for i in range(x.shape[0]):

    if i % 100 == 0:
        out[i] = out[i] = (x[i] - y[i])**2
    else:
        out[i] = out[i] = (y[i] - x[i])**2
return out

```

```

In [10]: branching_code(x32, y32)
find_instr(branching_code, keyword='subp', sig=0)

```

No instructions found

- Best to stick to pure math operations

SIMD and division

As a result, you would naturally assume a function like this would be OK:

```

In [11]: @jit(nopython=True)
def frac_diff1(x, y):
    out = np.empty_like(x)
    for i in range(x.shape[0]):
        out[i] = 2 * (x[i] - y[i]) / (x[i] + y[i])
    return out

```

```

In [12]: frac_diff1(x32, y32)

```

```

Out[12]: array([-0.66666667, -0.66662216, -0.66657776, ..., -0.400032,
               -0.40001604, -0.4], dtype=float32)

```

```

In [13]: %timeit frac_diff1(x32, y32)

```

25.6 µs ± 536 ns per loop (mean ± std. dev. of 7 runs, 10,000 loops each)

```

In [14]: find_instr(frac_diff1, keyword='subp', sig=0)

```

No instructions found

No instructions found ?!

The problem is that division by zero can behave in two different ways:

- In Python, division by zero raises an exception.
- In NumPy, division by zero results in a `NaN` or `inf`, like in C.

By default, Numba `@jit` follows the Python convention.

- A simple division operation `r = x / y` expands out into something like:

```

if y == 0:
    raise ZeroDivisionError()
else:
    r = x / y

```

- Autovectorizer gives up at this point 😞

Fortunately, Numba allows you to override the "error model" of the function if you don't want a `ZeroDivisionError` to be raised:

```
In [15]: @jit(nopython=True, error_model='numpy')
def frac_diff2(x, y):
    out = np.empty_like(x)
    for i in range(x.shape[0]):
        out[i] = 2 * (x[i] - y[i]) / (x[i] + y[i])
    return out
```

```
In [16]: frac_diff2(x32, y32)
```

```
Out[16]: array([-0.6666667 , -0.66662216, -0.66657776, ..., -0.400032  ,
               -0.40001604, -0.4          ], dtype=float32)
```

```
In [17]: find_instr(frac_diff2, keyword='subp', sig=0)
```

```
    vsubps    %ymm1, %ymm0, %ymm2
    vsubps    %ymm1, %ymm0, %ymm2
    vsubps    %ymm1, %ymm0, %ymm2
```

We have SIMD instructions again, we check the speed:

```
In [18]: %timeit frac_diff1(x32, y32)
%timeit frac_diff2(x32, y32)
%timeit frac_diff2(x64, y64)
```

```
26.8 µs ± 1.94 µs per loop (mean ± std. dev. of 7 runs, 10,000 loops each)
11.7 µs ± 269 ns per loop (mean ± std. dev. of 7 runs, 100,000 loops each)
15 µs ± 6.97 µs per loop (mean ± std. dev. of 7 runs, 1 loop each)
```

This is faster than the no-SIMD case, but there doesn't seem to be a speed benefit with `float32` inputs. What's going on?

The remaining issue is very subtle. We can see it if we look at a type-annotated version of the function:

```
In [19]: frac_diff2.inspect_types(pretty=True)
```

```
/opt/anaconda3/envs/phas0102/lib/python3.12/site-packages/numba/core/annota
tions/pretty_annotate.py:7: FutureWarning: The pretty_annotate functionalit
y is experimental and might change API
  warn("The pretty_annotate functionality is experimental and might change
API",
```

Out[19]: Function name: frac_diff2

in file:

/var/folders/cr/hx9slfzd61xdy7vghl6l64k80000gn/T/ipykernel_25014/2090011437.py
with signature: (Array(float32, 1, 'C', False, aligned=True), Array(float32, 1, 'C', False, aligned=True)) -> array(float32, 1d, C)

```

▶ 1: @jit(nopython=True, error_model='numpy')
    2: def frac_diff2(x, y):
▶ 3:     out = np.empty_like(x)
▶ 4:     for i in range(x.shape[0]):
▼ 5:         out[i] = 2 * (x[i] - y[i]) / (x[i] + y[i])
            $const100.2 = const(int, 2) :: Literal[int](2)
            $106binary_subscr.5 = getitem(value=x, index=i, fn=<built-in
function getitem>) :: float32
            $114binary_subscr.8 = getitem(value=y, index=i, fn=<built-in
function getitem>) :: float32
            $binop_sub118.9 = $106binary_subscr.5 - $114binary_subscr.8 ::
float32
            del $114binary_subscr.8
            del $106binary_subscr.5
            $binop_mul122.10 = $const100.2 * $binop_sub118.9 :: float64
            del $const100.2
            del $binop_sub118.9
            $130binary_subscr.13 = getitem(value=x, index=i, fn=<built-in
function getitem>) :: float32
            $138binary_subscr.16 = getitem(value=y, index=i, fn=<built-in
function getitem>) :: float32
            $binop_add142.17 = $130binary_subscr.13 + $138binary_subscr.16 ::
float32
            del $138binary_subscr.16
            del $130binary_subscr.13
            $binop_truediv146.18 = $binop_mul122.10 / $binop_add142.17 ::
float64
            del $binop_mul122.10
            del $binop_add142.17
            out[i] = $binop_truediv146.18 :: (Array(float32, 1, 'C', False,
aligned=True), int64, float64) -> none
            del i
            del $binop_truediv146.18
            jump 94

▶ 6:     return out

```

Function name: frac_diff2

in file:

/var/folders/cr/hx9slfzd61xdy7vghl6l64k80000gn/T/ipykernel_25014/2090011437.py
with signature: (Array(float64, 1, 'C', False, aligned=True), Array(float64, 1, 'C', False, aligned=True)) -> array(float64, 1d, C)

```

▶ 1: @jit(nopython=True, error_model='numpy')
    2: def frac_diff2(x, y):
▶ 3:     out = np.empty_like(x)
▶ 4:     for i in range(x.shape[0]):
▶ 5:         out[i] = 2 * (x[i] - y[i]) / (x[i] + y[i])
▶ 6:     return out

```

- `2 * (x[i] - y[i]...` is being promoted to float64
- rest of the calculation becomes float64.
- Conservative behaviour of Numba
- We can override this by forcing the type

```
In [20]: @jit(nopython=True, error_model='numpy')
def frac_diff3(x, y):
    out = np.empty_like(x)
    dt = x.dtype # Cast the constant using the dtype of the input
    for i in range(x.shape[0]):
        # Could also use np.float32(2) to always use same type, regardless of dtype
        out[i] = dt.type(2) * (x[i] - y[i]) / (x[i] + y[i])
    return out
```

```
In [21]: frac_diff3(x32, y32)
```

```
Out[21]: array([-0.6666667 , -0.66662216, -0.66657776, ..., -0.400032 ,
               -0.40001604, -0.4          ], dtype=float32)
```

```
In [22]: %timeit frac_diff3(x32, y32)
%timeit frac_diff3(x64, y64)
```

5.76 µs ± 213 ns per loop (mean ± std. dev. of 7 runs, 100,000 loops each)
 12.4 µs ± 357 ns per loop (mean ± std. dev. of 7 runs, 100,000 loops each)

Now lets compare the float32 speeds!

```
In [23]: print('-----')
print("No SIMD")
print('-----')
%timeit frac_diff1(x32, y32)
print('-----')
print("SIMD-np-error")
print('-----')
%timeit frac_diff2(x32, y32)
print('-----')
print("SIMD-dtype + np-error")
print('-----')
%timeit frac_diff3(x32, y32)
```

```
-----
No SIMD
-----
```

27.6 µs ± 3.53 µs per loop (mean ± std. dev. of 7 runs, 10,000 loops each)

```
-----
SIMD-np-error
-----
```

11.1 µs ± 491 ns per loop (mean ± std. dev. of 7 runs, 100,000 loops each)

```
-----
SIMD-dtype + np-error
-----
```

5.36 µs ± 240 ns per loop (mean ± std. dev. of 7 runs, 100,000 loops each)

We've hit 6x performance gain with relatively minor changes!

SIMD and Reductions

- Reductions are operations that aggregate or combine data along an axis (or axes)
- `sum`, `mean`, `std` are example
- Autovectorizer can also optimize reduction loops, but only with permission

Operation ordering

- Compilers are very careful not to reorder floating point instructions
- Floating point arithmetic is approximate
- it is not generally true for floating point numbers that:

$$(a + (b + c)) = ((a + b) + c)$$

Look at $10^{16} - 10^{16} + 1$ vs $10^{16} + 1 - 10^{16}$

```
In [31]: print(1e16 - 1e16 + 1)
1.0
```

```
In [32]: print(1e16 + 1 - 1e16)
0.0
```

- Most situations, round-off error is minimal, so this relation holds.
- In these situations, changing the order of operations is acceptable.
- Gain performance increase.

To allow reordering of operations, we need to tell Numba to enable `fastmath` optimizations:

```
In [26]: @jit(nopython=True)
def do_sum(A):
    acc = 0.
    # without fastmath, this loop must accumulate in strict order
    for i in range(A.size):
        acc += A[i]**2
    return acc

@jit(nopython=True, fastmath=True)
```

```
def do_sum_fast(A):
    acc = 0.
    # with fastmath, the reduction can be vectorized as floating point
    # reassociation is permitted.
    for i in range(A.size):
        acc += A[i]**2
    return acc
```

```
In [27]: do_sum(x32)
         find_instr(do_sum, keyword='mulp') # look for vector multiplication
```

No instructions found

```
In [28]: do_sum_fast(x32)
         find_instr(do_sum_fast, keyword='mulp')
```

```
vmulps  %xmm4, %xmm4, %xmm4
vmulps  %xmm5, %xmm5, %xmm5
vmulps  %xmm6, %xmm6, %xmm6
vmulps  %xmm7, %xmm7, %xmm7
vmulps  %xmm4, %xmm4, %xmm4
```

Now test it:

```
In [29]: %timeit do_sum(x32)
         %timeit do_sum_fast(x32)
```

```
16.1 µs ± 181 ns per loop (mean ± std. dev. of 7 runs, 100,000 loops each)
2.07 µs ± 19.7 ns per loop (mean ± std. dev. of 7 runs, 100,000 loops each)
```

Wow! lets be sure they are the same:

```
In [30]: do_sum(x32) == do_sum_fast(x32)
```

```
Out[30]: True
```

Result! 🤖

Conclusion

- Numba gives us the ability to autovectorize our code
- Need to be careful about how instructions are emitted to machine code
- Branching introduces difficulty in vectorising
- Careful about division behaviour
- Be aware about conservative promotion to `float64`
- If round-off error is minimal, reordering operations can increase performance!